

# Dynamic Programming

# 0/1 Knapsack Problem - Dynamic Programming Solution

► We need to generate table  $V(1 \dots n, 0 \dots W)$

1. where,  $n$  = number of objects.
2.  $W$  = capacity of knapsack.

To generate table  $V[i][j]$  use following steps:

Step-1: Make  $V[i][0] = 0$  for  $0 < i \leq n$

Step-2: if  $j < w_i$  then

$$V[i][j] = V[i - 1][j]$$

Step-3: if  $j \geq w_i$  then

$$V[i][j] = \max(V[i - 1][j], V[i - 1][j - w_i] + v_i)$$

# 0/1 Knapsack Problem - Dynamic Programming Solution

► Solve the following knapsack problem using dynamic programming technique.

1.  $W = 11$  and  $n = 5$

| Object $i$ | 1 | 2 | 3  | 4  | 5  |
|------------|---|---|----|----|----|
| $v_i$      | 1 | 6 | 18 | 22 | 28 |
| $w_i$      | 1 | 2 | 5  | 6  | 7  |

# 0/1 Knapsack Problem - Dynamic Programming Solution

|                                                             |                                                                                                                                                      |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| $V[i][0] = 0 \text{ for } 0 < i \leq n$                     | $V[3][6], \text{ here } i = 3, j = 6, w_3 = 5, v_3 = 18$<br>So, $j \geq w_3$<br>$V[3][6] = \max(V[2][6], V[2][1] + v_3)$<br>$= \max(7, 1 + 18) = 19$ |
| if $j < w_i$ then $V[i][j] = V[i - 1][j]$                   |                                                                                                                                                      |
| else $V[i][j] = \max(V[i - 1][j], V[i - 1][j - w_i] + v_i)$ |                                                                                                                                                      |

$j \longrightarrow$  Knapsack Capacity

| weight & value      | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---------------------|---|---|---|---|---|---|---|---|---|---|----|----|
| $w_1 = 1, v_1 = 1$  | 0 | 1 | 1 | 1 | 1 |   |   |   |   |   |    |    |
| $w_2 = 2, v_2 = 6$  | 0 | 1 | 6 | 7 | 7 |   |   |   |   |   |    |    |
| $w_3 = 5, v_3 = 18$ | 0 | 1 | 6 | 7 | 7 |   |   |   |   |   |    |    |
| $w_4 = 6, v_4 = 22$ | 0 | 1 | 6 | 7 | 7 |   |   |   |   |   |    |    |
| $w_5 = 7, v_5 = 28$ | 0 | 1 | 6 | 7 | 7 |   |   |   |   |   |    |    |

# 0/1 Knapsack Problem - Dynamic Programming Solution

► We can also find the objects to be carried in the knapsack as follows,

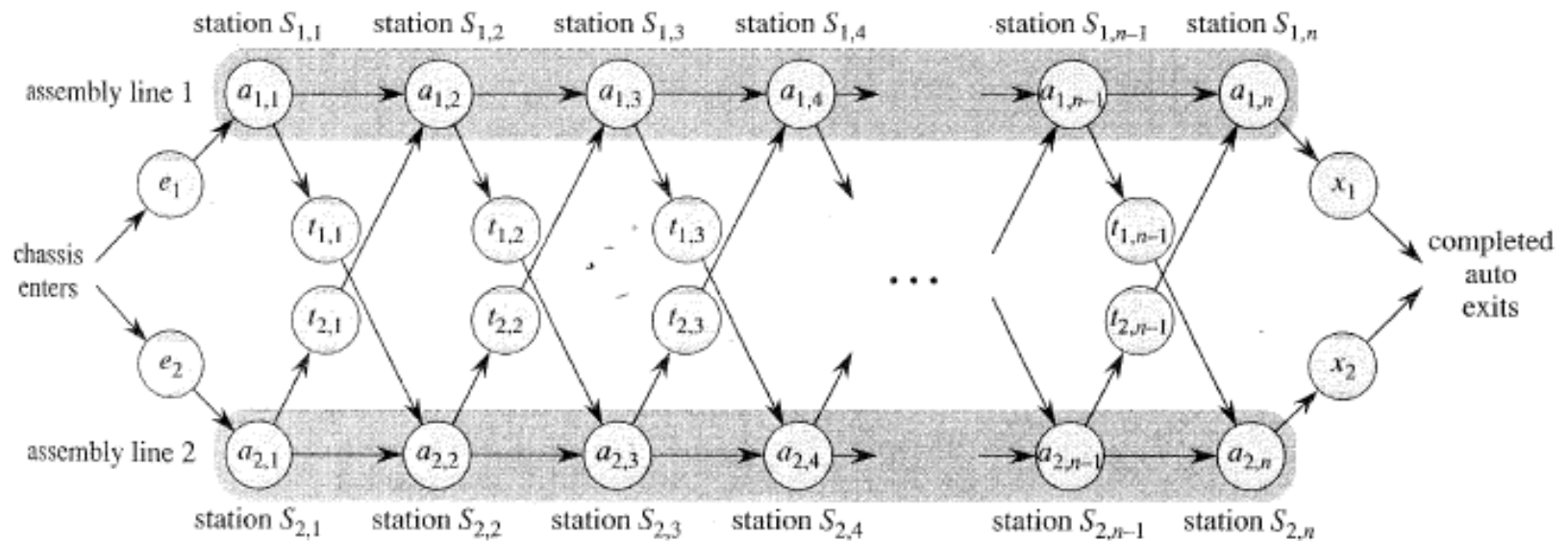
1. Start looking at  $V[5, 11] = V[4, 11] \Rightarrow$  So **not to include** object 5
2. Next go to  $V[4, 11] \neq V[3, 11]$  but  $V[4, 11] = V[3, 11 - w_4] + v_4 = V[3, 5] + 22$  So **include object 4**
3. Now go to  $V[3, 5] \neq V[2, 5]$  but  $V[3, 5] = V[2, 5 - w_3] + v_3 = V[2, 0] + 18$  So **include object 3**
4. Now  $V[2, 0] = V[1, 0]$  and  $V[1, 0] = V[0, 0] \Rightarrow$  So objects 2 and 1 are **not included**.

$$V[i][j] = \max(V[i-1][j], V[i-1][j - w_i] + v_i)$$

| weight & value      | 0 | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
|---------------------|---|---|---|---|---|----|----|----|----|----|----|----|
| $w_1 = 1, v_1 = 1$  | 0 | 1 | 1 | 1 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| $w_2 = 2, v_2 = 6$  | 0 | 1 | 6 | 7 | 7 | 7  | 7  | 7  | 7  | 7  | 7  | 7  |
| $w_3 = 5, v_3 = 18$ | 0 | 1 | 6 | 7 | 7 | 18 | 19 | 24 | 25 | 25 | 25 | 25 |
| $w_4 = 6, v_4 = 22$ | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 24 | 28 | 29 | 29 | 40 |
| $w_5 = 7, v_5 = 28$ | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 28 | 29 | 34 | 35 | 40 |

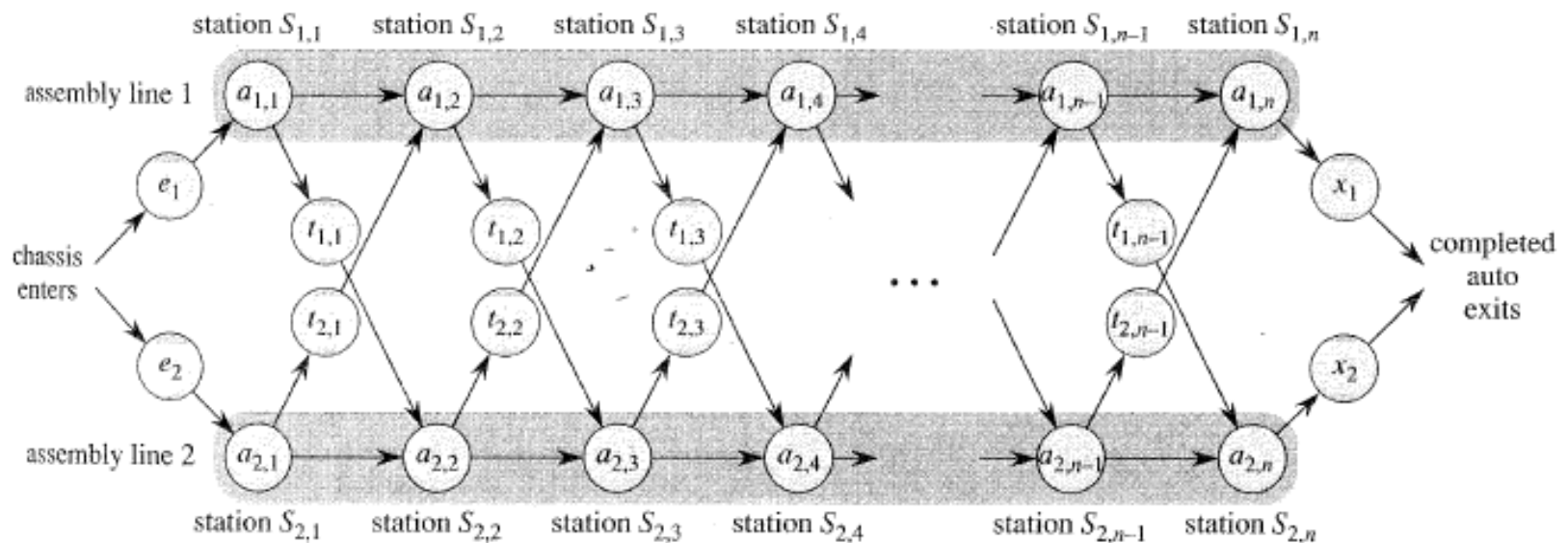
# Assembly Line Scheduling

- Automobile factory with two assembly lines
  - Each line has  $n$  stations:  $S_{1,1}, \dots, S_{1,n}$  and  $S_{2,1}, \dots, S_{2,n}$
  - Corresponding stations  $S_{1,j}$  and  $S_{2,j}$  perform the same function but can take different amounts of time  $a_{1,j}$  and  $a_{2,j}$
  - Entry times are:  $e_1$  and  $e_2$ ; exit times are:  $x_1$  and  $x_2$



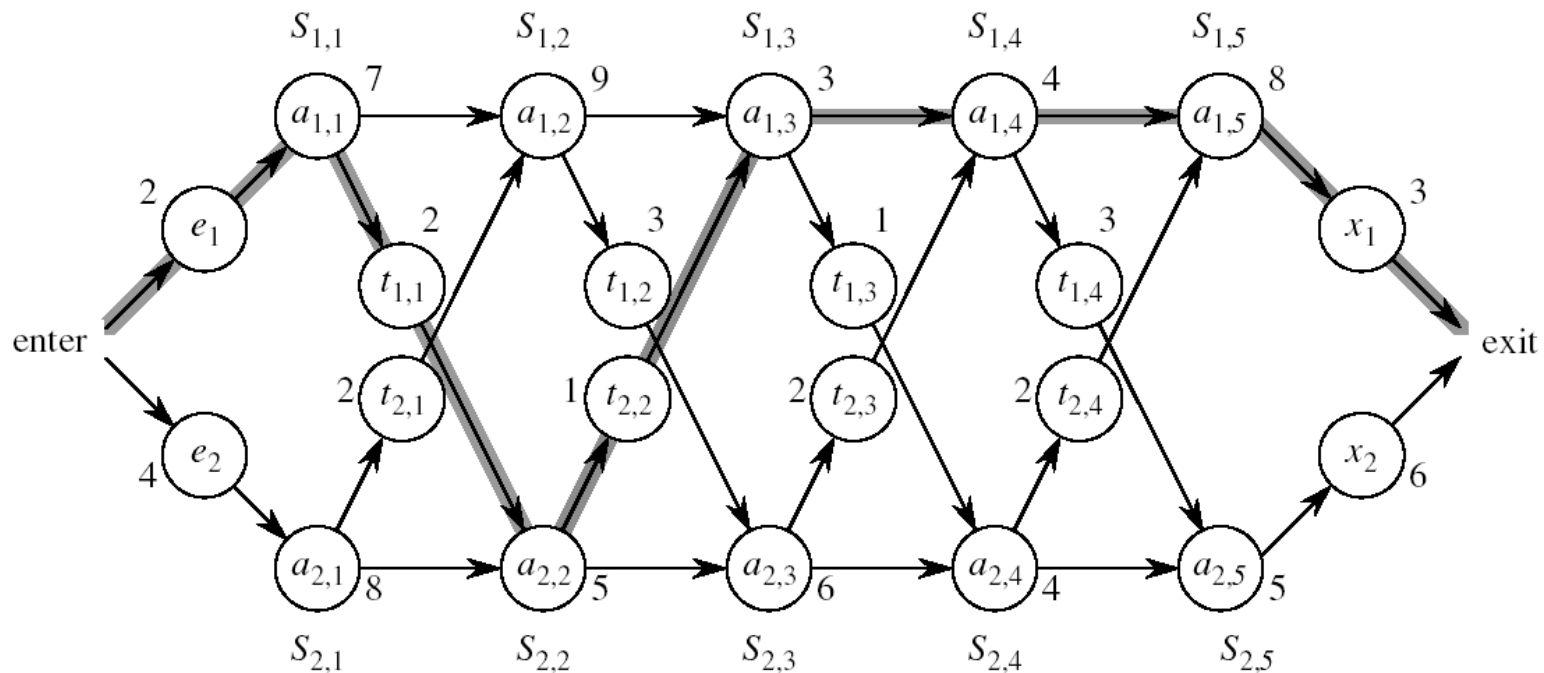
# Assembly Line Scheduling

- After going through a station, can either:
  - stay on same line at no cost, or
  - transfer to other line: cost after  $S_{i,j}$  is  $t_{i,j}$ ,  $j = 1, \dots, n - 1$



# Assembly Line Scheduling

- Problem:  
what stations should be chosen from line 1 and which from line 2 in order to **minimize the total time through the factory for one car?**



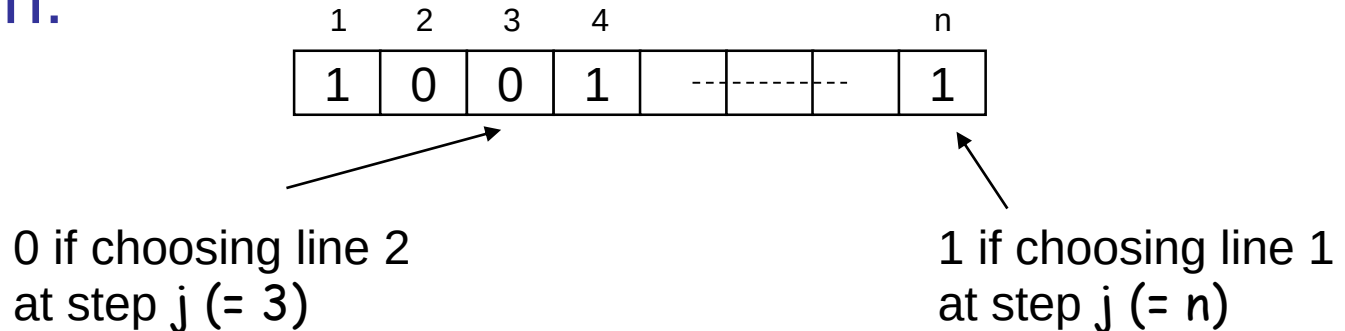


# One Solution

- Brute force

- Enumerate all possibilities of selecting stations
- Compute how long it takes in each case and choose the best one

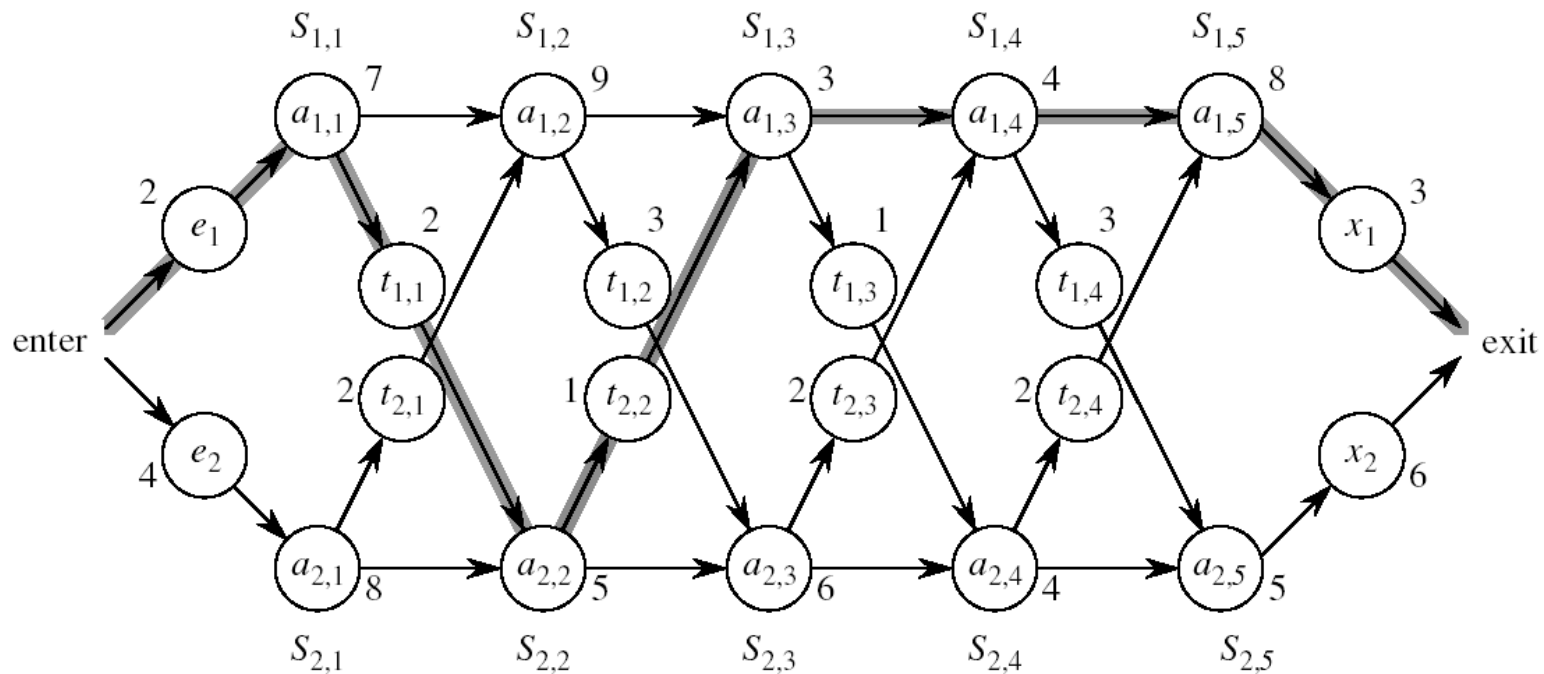
- Solution:



- There are  $2^n$  possible ways to choose stations
- Infeasible when  $n$  is large!!

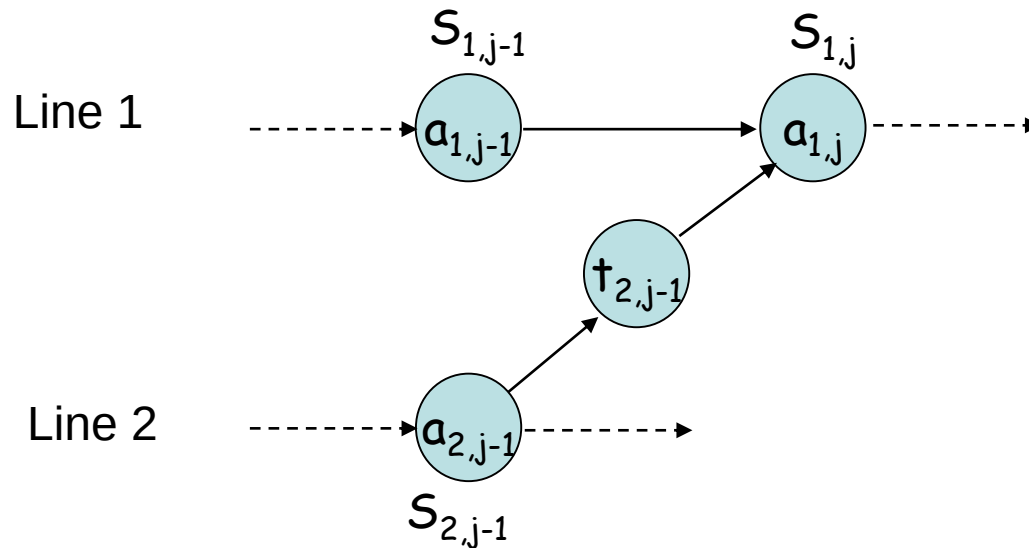
# 1. Structure of the Optimal Solution

- How do we compute the minimum time of going through a station?



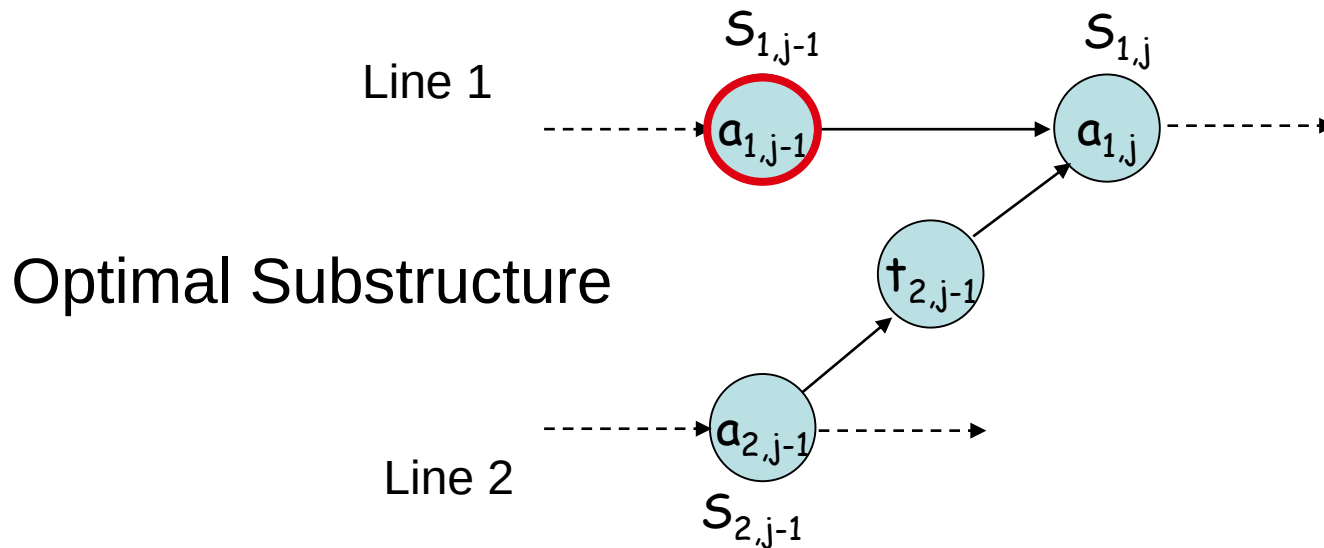
# 1. Structure of the Optimal Solution

- Let's consider all possible ways to get from the starting point through station  $S_{1,j}$ 
  - We have two choices of how to get to  $S_{1,j}$ :
    - Through  $S_{1,j-1}$ , then directly to  $S_{1,j}$
    - Through  $S_{2,j-1}$ , then transfer over to  $S_{1,j}$



# 1. Structure of the Optimal Solution

- Suppose that the fastest way through  $S_{1,j}$  is through  $S_{1,j-1}$ 
  - We must have taken a fastest way from entry through  $S_{1,j-1}$
  - If there were a faster way through  $S_{1,j-1}$ , we would use it instead
- Similarly for  $S_{2,j-1}$

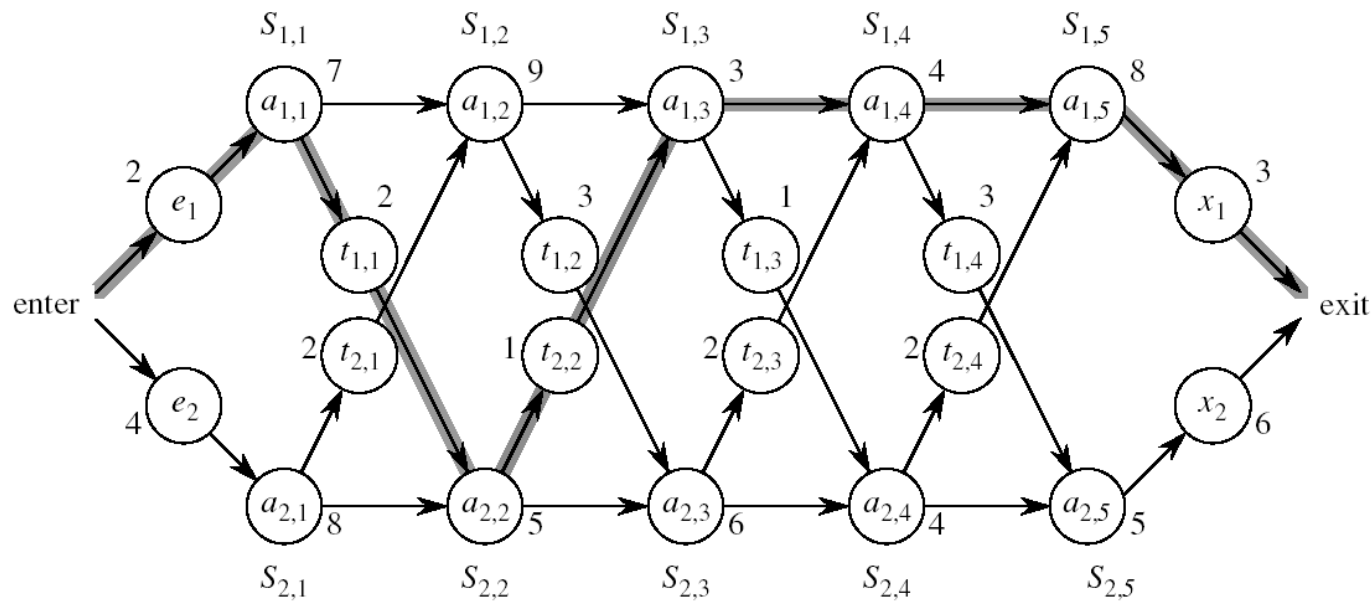


# Optimal Substructure

- **Generalization:** an optimal solution to the problem “*find the fastest way through  $S_{1,j}$* ” contains within it an optimal solution to subproblems: “*find the fastest way through  $S_{1,j-1}$  or  $S_{2,j-1}$* ”.
- This is referred to as the **optimal substructure** property
- We use this property to construct an optimal solution to a problem from optimal solutions to subproblems

## 2. A Recursive Solution

- Define the value of an optimal solution in terms of the optimal solution to subproblems

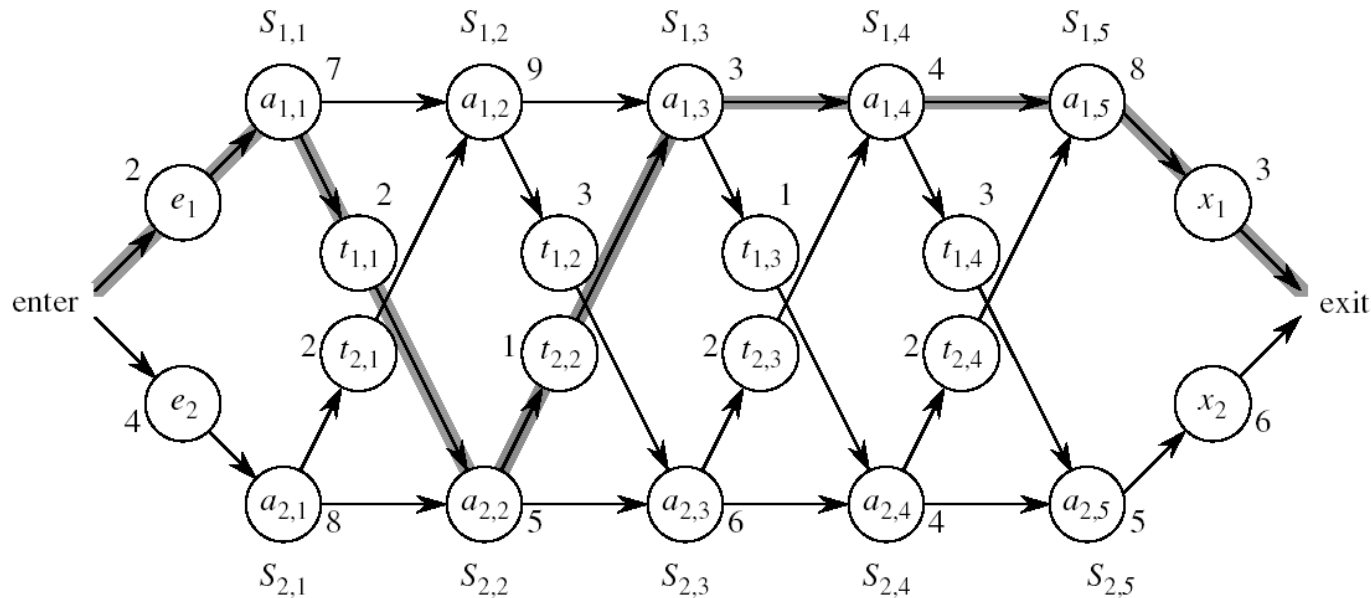


## 2. A Recursive Solution (cont.)

- Definitions:

- $f^*$  : the fastest time to get through the entire factory
- $f_i[j]$  : the fastest time to get from the starting point through station  $S_{i,j}$

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

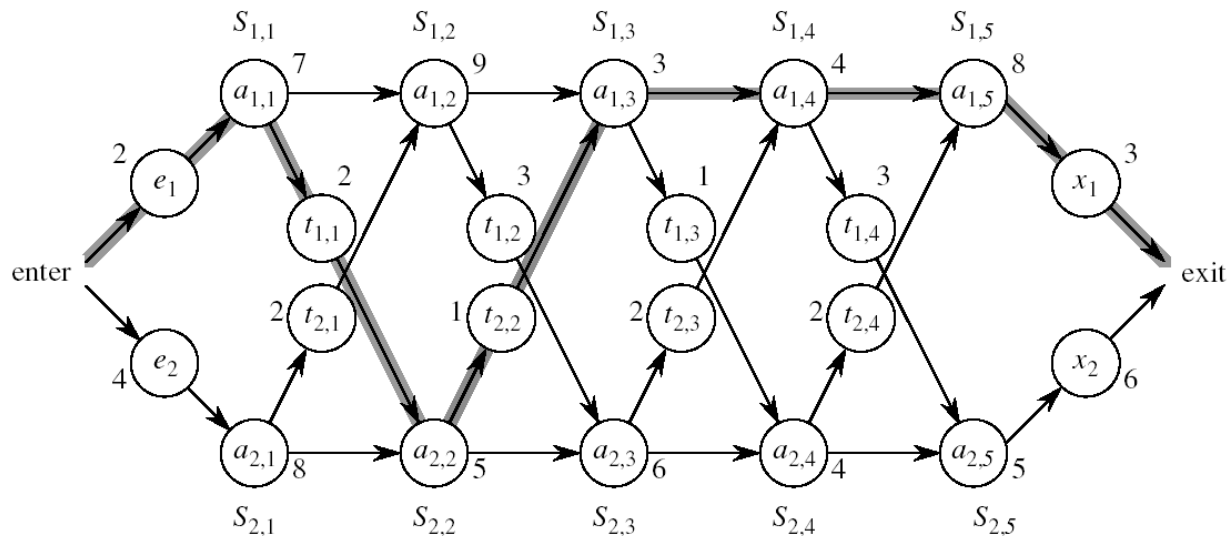


## 2. A Recursive Solution (cont.)

- Base case:  $j = 1, i=1,2$  (getting through station 1)

$$f_1[1] = e_1 + a_{1,1}$$

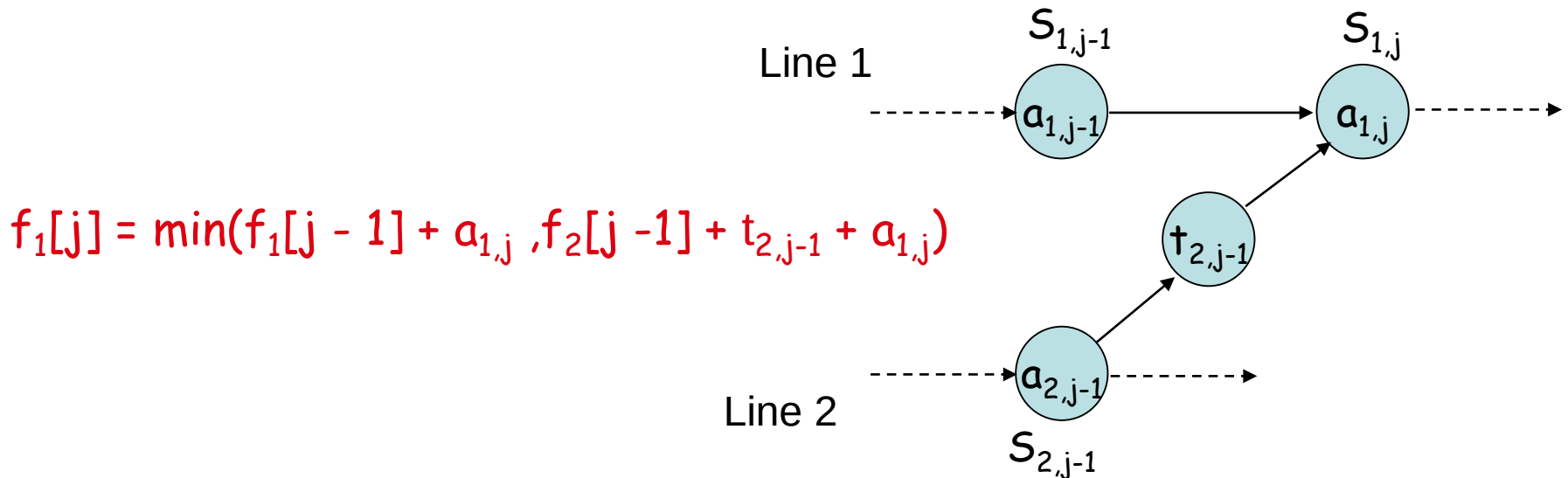
$$f_2[1] = e_2 + a_{2,1}$$





## 2. A Recursive Solution (cont.)

- General Case:  $j = 2, 3, \dots, n$ , and  $i = 1, 2$
- Fastest way through  $S_{1,j}$  is either:
  - the way through  $S_{1,j-1}$  then directly through  $S_{1,j}$ , or  
 $f_1[j-1] + a_{1,j}$
  - the way through  $S_{2,j-1}$ , transfer from line 2 to line 1, then through  $S_{1,j}$   
 $f_2[j-1] + t_{2,j-1} + a_{1,j}$



## 2. A Recursive Solution (cont.)

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j = 1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

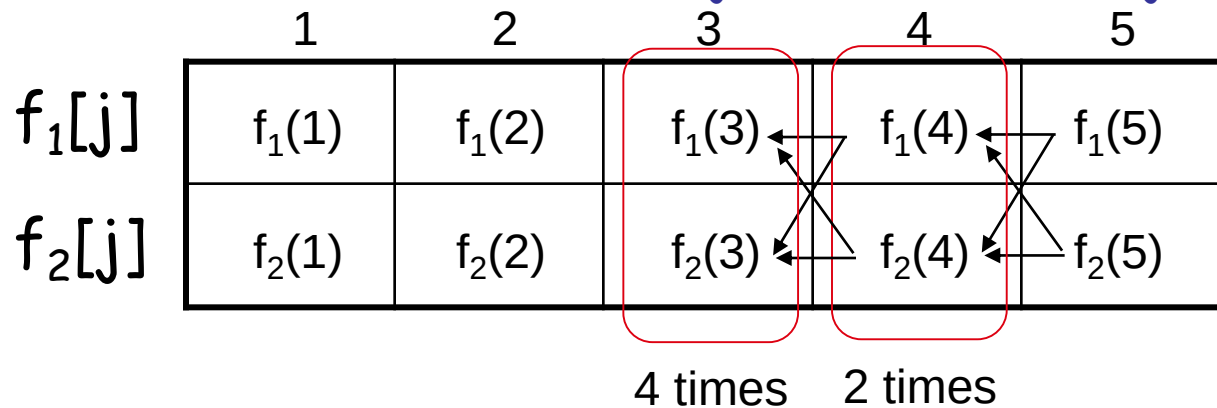
$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{if } j = 1 \\ \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) & \text{if } j \geq 2 \end{cases}$$

### 3. Computing the Optimal Solution

$$f^* = \min (f_1[n] + x_1, f_2[n] + x_2)$$

$$f_1[j] = \min(f_1[j - 1] + a_{1,j}, f_2[j - 1] + t_{2,j-1} + a_{1,j})$$

$$f_2[j] = \min(f_2[j - 1] + a_{2,j}, f_1[j - 1] + t_{1,j-1} + a_{2,j})$$



- Solving top-down would result in exponential running time

### 3. Computing the Optimal Solution

- For  $j \geq 2$ , each value  $f_i[j]$  depends only on the values of  $f_1[j - 1]$  and  $f_2[j - 1]$
- Idea: compute the values of  $f_i[j]$  as follows:

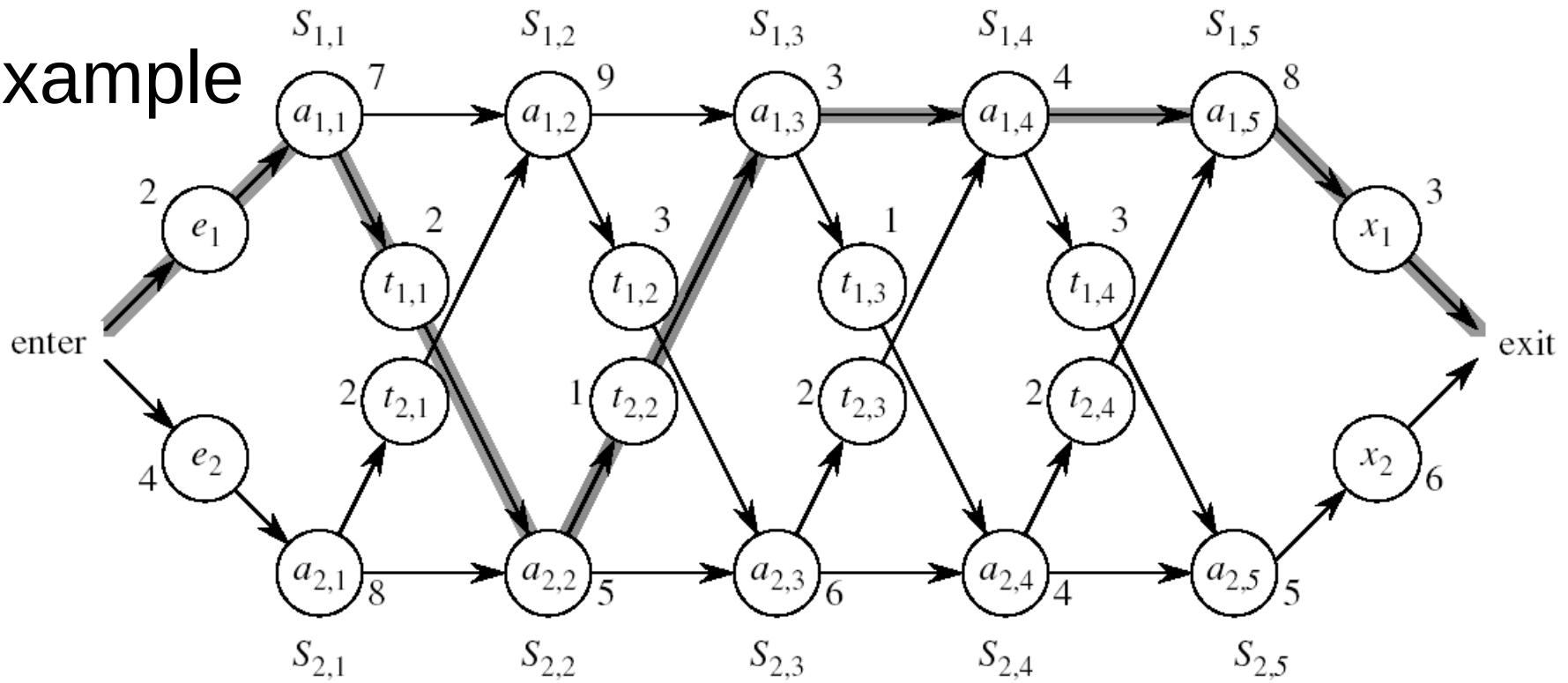
in increasing order of  $j$



|          | 1 | 2 | 3 | 4 | 5 |
|----------|---|---|---|---|---|
| $f_1[j]$ |   |   |   |   |   |
| $f_2[j]$ |   |   |   |   |   |

- **Bottom-up approach**
  - First find optimal solutions to subproblems
  - Find an optimal solution to the problem from the subproblems

# Example



$$f_1[j] = \begin{cases} e_1 + a_{1,1}, & \text{if } j = 1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

|          | 1  | 2                 | 3                 | 4                 | 5                 |
|----------|----|-------------------|-------------------|-------------------|-------------------|
| $f_1[j]$ | 9  | 18 <sup>[1]</sup> | 20 <sup>[2]</sup> | 24 <sup>[1]</sup> | 32 <sup>[1]</sup> |
| $f_2[j]$ | 12 | 16 <sup>[1]</sup> | 22 <sup>[2]</sup> | 25 <sup>[1]</sup> | 30 <sup>[2]</sup> |

$$f^* = 35^{[1]}$$

# FASTEST-WAY( $a, t, e, x, n$ )

```
1.  $f_1[1] \leftarrow e_1 + a_{1,1}$ 
2.  $f_2[1] \leftarrow e_2 + a_{2,1}$ 
3. for  $j \leftarrow 2$  to  $n$ 
4.   do if  $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
5.     then  $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$ 
6.          $l_1[j] \leftarrow 1$ 
7.     else  $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
8.          $l_1[j] \leftarrow 2$ 
9.   if  $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
10.    then  $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$ 
11.         $l_2[j] \leftarrow 2$ 
12.    else  $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
13.         $l_2[j] \leftarrow 1$ 
```

Compute initial values of  $f_1$  and  $f_2$

**$O(N)$**

Compute the values of  $f_1[j]$  and  $l_1[j]$

Compute the values of  $f_2[j]$  and  $l_2[j]$

# FASTEST-WAY( $a, t, e, x, n$ ) (cont.)

|                                         |   |                                                                         |
|-----------------------------------------|---|-------------------------------------------------------------------------|
| 14. if $f_1[n] + x_1 \leq f_2[n] + x_2$ | } | Compute the values of<br>the fastest time through the<br>entire factory |
| 15.     then $f^* = f_1[n] + x_1$       |   |                                                                         |
| 16. $l^* = 1$                           |   |                                                                         |
| 17.     else $f^* = f_2[n] + x_2$       |   |                                                                         |
| 18. $l^* = 2$                           |   |                                                                         |

# 4. Construct an Optimal Solution

*Alg.:* PRINT-STATIONS( $l, n$ )

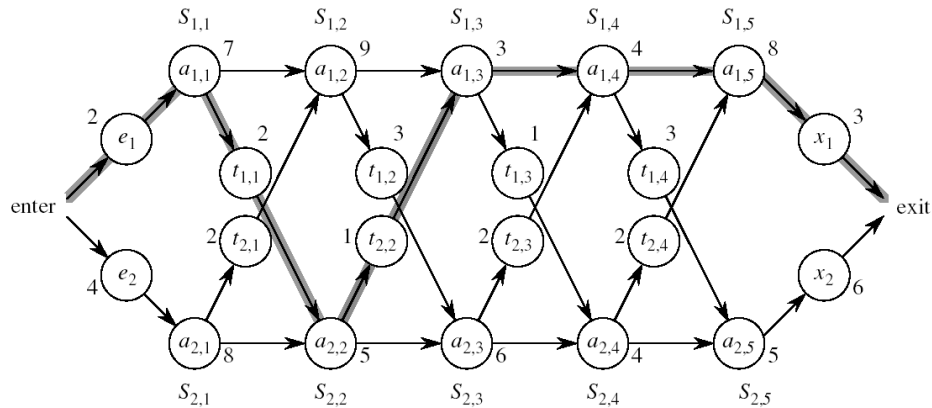
$i \leftarrow l^*$

print "line "  $i$  ", station "  $n$

for  $j \leftarrow n$  downto 2

do  $i \leftarrow l_i[j]$

print "line "  $i$  ", station "  $j - 1$



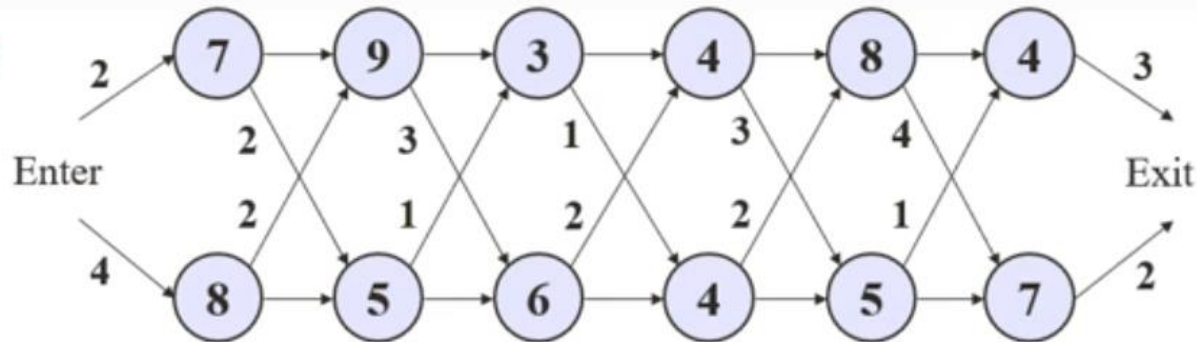
|                 | 1  | 2                 | 3                 | 4                 | 5                 |
|-----------------|----|-------------------|-------------------|-------------------|-------------------|
| $f_1[j]/l_1[j]$ | 9  | 18 <sup>[1]</sup> | 20 <sup>[2]</sup> | 24 <sup>[1]</sup> | 32 <sup>[1]</sup> |
| $f_2[j]/l_2[j]$ | 12 | 16 <sup>[1]</sup> | 22 <sup>[2]</sup> | 25 <sup>[1]</sup> | 30 <sup>[2]</sup> |

$l^* = 1$



# Assembly line Scheduling Problem: Dynamic Programming

**Example:**



$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j = 1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

$$f^* = 38$$

$$l^* = 1$$

|          | 1  | 2  | 3  | 4  | 5  | 6  |
|----------|----|----|----|----|----|----|
| $f_1[j]$ | 9  | 18 | 20 | 24 | 32 | 35 |
| $f_2[j]$ | 12 | 16 | 22 | 25 | 30 | 37 |

|          | 1 | 2 | 3 | 4 | 5 | 6 |
|----------|---|---|---|---|---|---|
| $l_1[j]$ | 1 | 1 | 2 | 1 | 1 | 2 |
| $l_2[j]$ | 2 | 1 | 2 | 1 | 2 | 2 |

# Assembly line Scheduling Problem: Dynamic Programming

